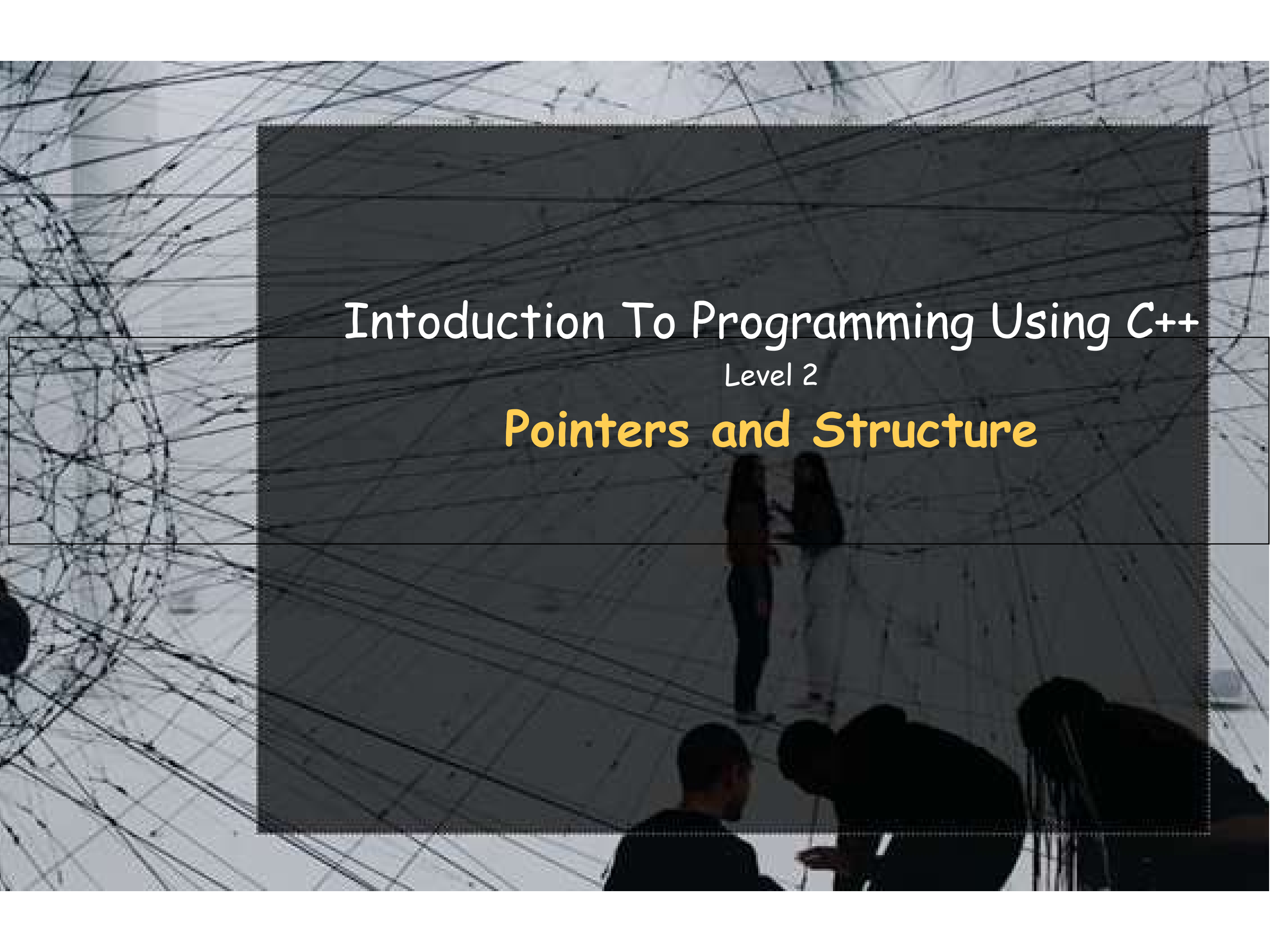




Introduction To Programming Using C++ Level 2

Pointers and Structure



Introduction To Programming Using C++

Level 2

Pointers and Structure

Definition

Pointer to a Structure

-  A pointer can point to a Structure object in C++.

-  When accessing a Structure member through a pointer, we can use the arrow operator (->) instead of the dot operator (.).

Arrow Operator

-  The arrow operator (->) is used to access a member of a Structure through a pointer.

-  The following two expressions produce the same result:
(*ptr).Name
ptr->Name

Explanation

Structure

 A Structure is a user-defined data type that can contain multiple members of different types.

 For example, an employee Structure can contain a Name and a Salary.

Accessing Members Directly

 When we have a normal Structure object, we use the dot operator (.).

 Example:

Employee1.Name

Employee1.Salary

Accessing Members Through a Pointer

 When we have a pointer to a Structure, we use the arrow operator (->).

 Example:

ptr->Name

ptr->Salary

Procedures

1. Define a Structure.
2. Create a Structure object.
3. Assign values to its members.
4. Create a pointer to the Structure.
5. Store the address of the Structure object in the pointer.
6. Use `->` to access the Structure members through the pointer.
7. Alternatively, use `(*ptr).member` to access the same members.

💡 Example

◆ Pointer to a Structure

```
struct stEmployee
{
    string Name;
    float Salary;
};
int main()
{
    stEmployee Employee1, *ptr;
    Employee1.Name = "Sara Alterkawi";
    Employee1.Salary = 5000;
    ptr = &Employee1;
    cout << ptr->Name << endl;
    cout << ptr->Salary << endl;
    cout << (*ptr).Name << endl;
    cout << (*ptr).Salary << endl;
    return 0;
}
```

➡ Result

```
Sara Alterkawi
5000
Sara Alterkawi
```

Note

- ◆ Use the dot operator (.) when working directly with a Structure object:
Employee1.Name
- ◆ Use the arrow operator (->) when working with a pointer to a Structure:
ptr->Name
- ◆ Parentheses are important when writing the longer form:
(*ptr).Name
- ◆ This is not the same as:
*ptr.Name
- ◆ The expression:
(*ptr).Name
means:
 - ▣ Dereference ptr.
 - ▣ Then access Name.

⊘ Warning

◆ Do not confuse the arrow operator (->) with the less-than and minus operators.

◆ Also, do not use . directly on a pointer:

ptr.Name // ✗ Incorrect

◆ Use:

ptr->Name // ✓ Correct

◆ Or:

(*ptr).Name // ✓ Correct



Comparison

▨ Structure Object

- ✓ Use the dot operator (.).
- ✓ Example:
Employee1.Name
- ✓ The object itself is used directly.

▤ Structure Pointer

- ✓ Use the arrow operator (->).
- ✓ Example:
ptr->Name
- ✓ The pointer must point to a valid Structure object.

✦ Summary

- ✓ A pointer can point to a Structure object.
- ✓ Use . when accessing a member through a normal Structure object.
- ✓ Use -> when accessing a member through a pointer to a Structure.
- ✓ ptr->Name is equivalent to (*ptr).Name.
- ✓ The arrow operator combines dereferencing and member access into one shorter syntax.

Thank you 😊